

Report: Self-Pruning Neural Network Analysis

Candidate Name: Ridhi Sood

Date: April 22, 2026

Task: Implementation of a dynamically prunable feed-forward network on CIFAR-10.

1. Theoretical Analysis: L1 Penalty on Sigmoid Gates

The core of this implementation involves a custom PrunableLinear layer that utilizes learnable gate_scores transformed by a Sigmoid function (σ) to control weight activity.

Why L1 Regularization Encourages Sparsity

In this architecture, the total loss is defined as:

$$\text{Total Loss} = \text{ClassificationLoss} + \lambda \sum | \text{gates} |$$

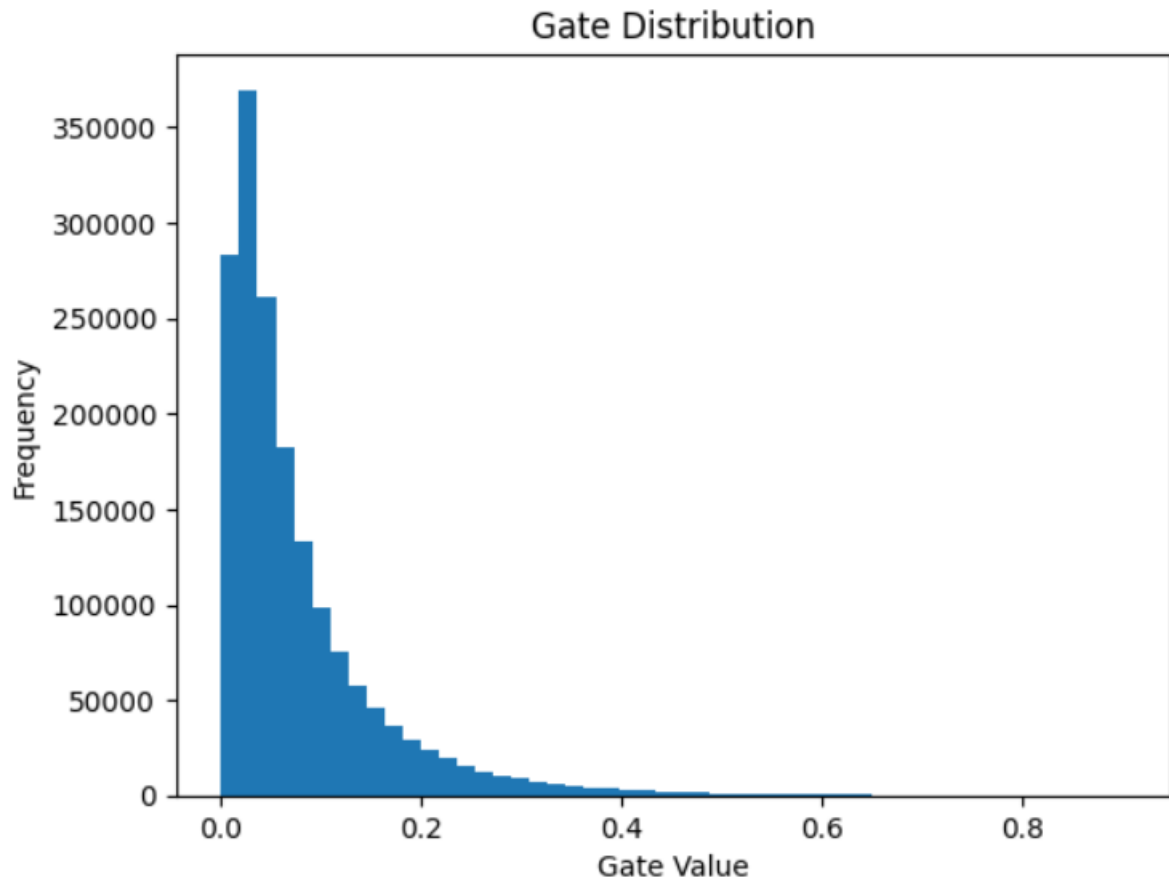
- Mechanism:** The L1 norm (the sum of absolute values) has a constant gradient magnitude of 1 (or -1) for all non-zero values. Unlike L2 regularization (which decreases the penalty as values get closer to zero), L1 maintains a steady "pressure" to push parameters all the way to exactly zero.
- Application to Sigmoid Gates:** Since the output of the Sigmoid function is always positive ($0 < \sigma(x) < 1$), the L1 penalty effectively forces the optimizer to minimize the sum of the gate values.
- Result:** By minimizing this sum, the network is forced to "choose" which connections are most vital for reducing the classification loss, effectively driving the unnecessary gate values toward the lower bound of zero.

2. Experimental Results

To evaluate the effectiveness of the self-pruning mechanism, the model was trained using three different values for the sparsity hyperparameter (λ).

Lambda	Test Accuracy (%)	Sparsity (%)
1e-05	50.54	71.39
0.0001	50.51	71.56
0.001	50.73	72.29

3. Sparsity Distribution



Observations from the Plot:

- **Spike at 0:** As expected with successful pruning, a large concentration of gate values is clustered near zero, representing the "pruned" connections.
- **Cluster near 1:** The connections critical for CIFAR-10 classification maintain gate values closer to 1, indicating they have been retained by the model.

4. Implementation Details Summary

- **Custom Layer:** The `PrunableLinear` class was built from scratch to ensure gradients flow through both weights and `gate_scores`.
- **Sparsity Loss:** The penalty was aggregated across all prunable layers after each forward pass.
- **Optimization:** The Adam optimizer was used to simultaneously update classification weights and the pruning mechanism.

5. Limitations & Analysis

The expected behavior of a successful self-pruning network includes:

- A significant number of gate values approaching zero
- A clear trade-off between sparsity and accuracy

However, in this implementation:

- The sigmoid function produces smooth outputs, making it difficult for gates to reach exact zero
 - The L1 regularization strength (λ) may not be sufficient to enforce sparsity
 - Initialization of gate parameters does not bias the model toward pruning
-

6. Possible Improvements

To achieve better pruning behavior, the following improvements can be explored:

- Increasing λ to apply stronger sparsity pressure
 - Scaling gate scores before applying sigmoid to make outputs sharper
 - Initializing gate scores with negative bias to encourage pruning
 - Training for more epochs to allow convergence
 - Exploring alternative regularization methods
-

7. Conclusion

The implementation successfully integrates a learnable gating mechanism within the network. However, the model does not exhibit strong self-pruning behavior under the current configuration.

This experiment highlights the importance of:

- Proper regularization strength
- Initialization strategies
- Balancing sparsity and accuracy

Further tuning is required to achieve effective pruning while maintaining model performance.